

คำอธิบายแต่ละขั้นตอน

โมเดล DecisionTreeRegressor

การพยากรณ์ปริมาณน้ำฝน — TMD Cluster 6

Notebook นี้มี 23 Cell | เตรียมข้อมูล 17 ขั้นตอน | สร้างโมเดล 1 ขั้นตอน | ประเมินผล + แสดงกราฟ 5 ขั้นตอน
Pipeline ทุกขั้นตอนเหมือนกับ LSTM_V23_11 เพื่อให้เปรียบเทียบผลได้อย่างยุติธรรม

ส่วนที่ 1 ตั้งค่าและโหลดข้อมูล

ขั้นตอนที่ 1 Import Libraries	
ทำอะไร	นำเข้า library ที่จำเป็นทั้งหมด: os, copy, random, pickle, time, warnings — ใช้งานทั่วไป numpy, pandas — จัดการ array และ DataFrame matplotlib — วาดกราฟ dateutil.relativedelta — คำนวณช่วงวันที่ sklearn.preprocessing.StandardScaler — normalize ข้อมูล sklearn.tree.DecisionTreeRegressor — โมเดลหลัก sklearn.metrics — คำนวณ MSE, R²
ทำเพื่ออะไร	Python ต้องประกาศ import ก่อนใช้งาน การรวม import ไว้ต้นไฟล์ทำให้เห็น dependency ทั้งหมดในที่เดียว และแก้ปัญหา missing library ได้ง่าย

ขั้นตอนที่ 2 กำหนด Hyperparameters

ทำอะไร	กำหนดค่าพารามิเตอร์หลักสองกลุ่ม: ● กลุ่มเหมือน LSTM: algo, window_size=24, horizon=12, cluster=6, k_fold_num=2 ● กลุ่ม Decision Tree: max_depth=None, min_samples_split=2, min_samples_leaf=1, random_state=0
ทำเพื่ออะไร	รวมค่าคงที่ไว้จุดเดียว ทำให้เปลี่ยนได้ง่ายโดยไม่ต้องตามแก้ทีละที่ window_size=24 และ horizon=12 เหมือน LSTM เพื่อให้ input/output format ตรงกันในการเปรียบเทียบ max_depth=None = ต้นไม้เติบโตจนสุด (ทำนาย training data ได้ 100%) — ปรับเป็น 5-15 เพื่อลด overfit ได้

ขั้นตอนที่ 3 กำหนด Path ไฟล์	
ทำอะไร	กำหนด base_path และสร้าง path ของไฟล์ทั้ง 4: X_variable(Index)_operation_v4.csv — climate indices Y_variable(Rainfall).csv — ข้อมูลฝน Node_table_TMD.csv — metadata สถานี lookup_table_reanalysis_v5.csv — ตาราง feature lookup
ทำเพื่ออะไร	แยก path ออกจาก logic ทำให้ย้าย environment ได้ง่าย เช่น เปลี่ยน base_path บรรทัดเดียวก็รันบน Colab ได้เลย

ขั้นตอนที่ 4 โหลดข้อมูลดิบ (= LSTM Cell 7+8)	
ทำอะไร	อ่านไฟล์ CSV ทั้ง 4 เข้ามาเป็น DataFrame แปลง DATE ของ X ให้เป็น format 'YYYY-MM-01' ให้ตรงกับ Y เปลี่ยนชื่อคอลัมน์ 'code' ใน Y เป็น 'Node' นับจำนวน feature สำหรับ cluster 6 จาก lookup_table (num_feature=8)
ทำเพื่ออะไร	ไฟล์เหล่านี้คือแหล่งข้อมูลทั้งหมดของระบบ — ต้องโหลดก่อนทุกอย่าง lookup_table ทำให้ code นี้รองรับ cluster อื่นได้ทันทีโดยแค่เปลี่ยน cluster=6 เป็นค่าอื่น

ส่วนที่ 2 เตรียมข้อมูล

ขั้นตอนที่ 5 สร้าง Lag Features (= LSTM Cell 9)

ทำอะไร	เรียก <code>apply_time_lag()</code> เพื่อเพิ่มคอลัมน์ lag ให้ X ก่อน normalize: MEIV2_lag1, MEIV2_lag2, MEIV2_lag3 RMM_AMPLITUDE_lag1 PHASE_lag1 PDO_lag1, PDO_lag2 ONI_lag1, ONI_lag2 DMI_lag1, DMI_lag2 BSISO1_lag1 BSISO1-Phase_lag1 SWM_lag1 หลังจากนี้ X มี 24 คอลัมน์ (10 เดิม + 14 lag)
ทำเพื่ออะไร	ดัชนีภูมิอากาศหลายตัวส่งผลต่อฝนในประเทศไทยแบบ delayed เช่น ENSO (ONI) ใช้เวลา 1-2 เดือนกว่าจะมีผล การสร้าง lag ก่อน normalize สำคัญมาก เพราะ StandardScaler ต้องเห็นค่า lag จริงเพื่อคำนวณ mean และ std ที่ถูกต้อง เหมือนกับ LSTM Cell 9 ทุกประการ ทำให้ feature set ตรงกันในการเปรียบเทียบ

ขั้นตอนที่ 6 Data Cleansing — เติม Missing ของฝน (= LSTM Cell 11)

ทำอะไร	คำนวณค่า median ของแต่ละสถานีแยกตามเดือน (เดือน 1-12) จากทุกปีที่มีข้อมูล วน loop ตรวจทุก cell: ถ้าพบ NaN ให้แทนที่ด้วย median ของเดือนนั้น
ทำเพื่ออะไร	โมเดล ML ทุกชนิดไม่สามารถรับค่า NaN เป็น input ได้ ต้องเติมให้ครบก่อน ใช้ monthly median แทน global mean เพราะฝนมี seasonality ชัดเจน — เดือนสิงหาคมฝนมากกว่าเดือนมกราคมมาก ค่า median รายเดือนสะท้อน pattern นี้ได้ดีกว่า เหมือน LSTM ทุกบรรทัด ทำให้ชุดข้อมูลที่ป้อนโมเดลทั้งสองตรงกัน

ขั้นตอนที่ 7 Filter Date Range ครั้งที่ 1 (= LSTM Cell 13)

ทำอะไร	กำหนด <code>start_training_date = '1982-01-01'</code> หา <code>max_date_possible = min(max ของ X, max ของ Y)</code> ตัด X และ Y ให้เหลือเฉพาะช่วง 1982-01-01 ถึง <code>max_date_possible</code>
--------	---

ทำเพื่ออะไร	ข้อมูล X ก่อนปี 1982 มีค่า NaN จำนวนมาก (ดัชนีบางตัวยังไม่มีประวัติ) ไม่เหมาะสมสำหรับเทรน การกำหนดจุดสิ้นสุดเป็น min ของ max ทั้งสองป้องกัน index mismatch เมื่อ merge ในภายหลัง
-------------	--

ขั้นตอนที่ 8 Filter Date Range ครั้งที่ 2 (= LSTM Cell 14)

ทำอะไร	LSTM Cell 14 รันต่อจาก Cell 13 โดย reset start_training_date = Y_rainfall_clean['DATE'].min() แล้ว filter ซ้ำด้วยช่วงเวลาใหม่นี้
ทำเพื่ออะไร	Cell 14 ใน LSTM เป็น cell แยกที่รันต่อกัน ทำให้ start อาจเปลี่ยนถ้า Y มีวันแรกหลังจาก 1982 ต้องทำเหมือนกันทุกประการเพื่อให้ได้ข้อมูลช่วงเดียวกับ LSTM — ในกรณีนี้ผลลัพธ์เหมือนเดิม (519 แถว)

ขั้นตอนที่ 9 ตั้ง DATE เป็น Index (= LSTM Cell 17+19)

ทำอะไร	เรียก convert_columns_to_index_columns() เพื่อแปลงคอลัมน์ DATE ให้กลายเป็น index ของ DataFrame ทำทั้ง X_index_df_ready (519×24) และ Y_rainfall_df_ready (519×132)
ทำเพื่ออะไร	การใช้ DATE เป็น index ทำให้ slice ข้อมูลตามช่วงเวลาได้สะดวกด้วย syntax df['2019-01':'2021-12'] ทำให้ code แบ่ง Train/Val/Test ในขั้นถัดไปอ่านง่ายและไม่เกิดข้อผิดพลาด off-by-one

ขั้นตอนที่ 10 Normalize ด้วย StandardScaler (= LSTM Cell 22+25)

ทำอะไร	สร้าง StandardScaler() แล้วเรียก fit_transform() บน X_index_df_ready → X_index_normalized_df สร้าง StandardScaler() ใหม่แล้ว fit_transform() บน Y_rainfall_df_ready → Y_rainfall_normalized_df
--------	---

	Restore date index กลับด้วย <code>og_date_range = X_index_interval_date['DATE']</code> (เพราะ <code>fit_transform</code> reset index)
ทำเพื่ออะไร	StandardScaler แปลงข้อมูลเป็น z-score: $z = (x - \text{mean}) / \text{std}$ ทำให้ทุก feature มี <code>mean=0, std=1</code> ดัชนีภูมิอากาศแต่ละตัวมี scale ต่างกันมาก เช่น ONI อยู่ช่วง -3 ถึง +3 แต่ PDO อาจแตกต่างกันมาก — ถ้าไม่ normalize โมเดลจะให้น้ำหนักกับ feature ที่มีค่าสูงกว่าโดยอัตโนมัติ fit บน dataset ทั้งหมด (ไม่แยก train/test ก่อน) เหมือน LSTM ทำให้ scale ตรงกันสำหรับการเปรียบเทียบ

ขั้นตอนที่ 11 เพิ่ม Month_sin / Month_cos (= LSTM Cell 28+30)	
ทำอะไร	$\text{Month_sin} = \sin(2\pi \times X.\text{index.month} / 12)$ $\text{Month_cos} = \cos(2\pi \times X.\text{index.month} / 12)$ เพิ่มหลัง normalize — ทำให้ X มี 26 คอลัมน์ Update <code>n_feature_climate = 26</code> และ <code>feature_climate</code>
ทำเพื่ออะไร	เดือนเป็นตัวเลขวนรอบ (circular): เดือน 12 และเดือน 1 ติดกัน แต่ตัวเลข 12 กับ 1 ห่างกันมาก การแปลงเป็น sin/cos แก้ปัญหานี้ — เดือน 12 และ 1 จะอยู่ใกล้กันในพื้นที่ sin/cos เพิ่มหลัง normalize เพราะ sin/cos อยู่ในช่วง -1 ถึง +1 อยู่แล้ว ไม่ต้อง normalize ซ้ำ

ขั้นตอนที่ 12 สร้าง Per-Station Table (= LSTM Cell 29)	
ทำอะไร	วน loop ทุกสถานี (132 สถานี): copy <code>X_index_normalized_df</code> แล้วเพิ่มคอลัมน์ <code>rainfall</code> ของสถานีนั้น เก็บใน <code>list_table</code> — list ที่มี 132 DataFrames แต่ละอันมี 27 คอลัมน์ (26 climate + 1 rainfall) คอลัมน์สุดท้ายของทุก table คือ <code>rainfall</code> ของสถานีนั้น
ทำเพื่ออะไร	LSTM ออกแบบให้ทำนายทุกสถานีพร้อมกันผ่าน HeteroData graph

Decision Tree ต้องการ tabular data แยกต่อสถานี การมี DataFrame แยกต่อสถานีทำให้ sliding window ในขั้นถัดไปดึงข้อมูลแต่ละสถานีได้โดยตรง

ขั้นตอนที่ 13 Feature Selection (= LSTM Cell 44+48)

ทำอะไร

Cell 44: วน loop ค้นหาใน feature_climate ว่า feature ที่ lookup_table กำหนดไว้อยู่ที่ตำแหน่ง index ไດ

- ถ้า lag=0: หาดตรงๆ จาก feature_climate
- ถ้า lag≠0: หา 'feature_name_lagN' จาก feature_climate

ผล: selected_index = [1, 3, 5, 8, 9, 17, 24, 25] (8 features)

Cell 48: สร้าง position vector ขนาด 27 (26+1) กำหนด 1 ที่ตำแหน่งที่เลือก + position_remain=[1] สำหรับ rainfall

ผล: position_all_index = [1,3,5,8,9,17,24,25,26] — 9 คอลัมน์ต่อสถานี

ทำเพื่ออะไร

ไม่ใช่ทุก feature ที่เหมาะกับทุก cluster — lookup_table เลือกไว้แล้วจากการวิเคราะห์ก่อนหน้านี้

Cluster 6 ใช้: RMM_AMPLITUDE, BSISO1, DMI, SWM, NE, ONI_lag1, Month_sin, Month_cos

การตัด feature ที่ไม่เกี่ยวลด noise ช่วยให้โมเดลโฟกัสที่ signal จริง

ขั้นตอนที่ 14 กำหนดช่วงวันที่ (= LSTM Cell 38)

ทำอะไร

คำนวณ start_interval_train1 = min(Y_rainfall_normalized_df.index) = 1982-01-01

คำนวณช่วงวันที่ทั้ง 6 ช่วงด้วย relativedelta:

- Fold 1 Train: 1982-01 → 2019-12 (456 เดือน)
- Fold 1 Val: 2018-01 → 2020-12 (36 เดือน overlap กับ train 2 ปี)
- Fold 1 Test: 2019-01 → 2021-12 (36 เดือน)
- Fold 2 Train: 1982-01 → 2022-01 (481 เดือน)
- Fold 2 Val: 2021-01 → 2023-12 (36 เดือน)
- Fold 2 Test: 2022-01 → 2024-12 (36 เดือน)

ทำเพื่ออะไร	Time-series ไม่สามารถ shuffle ข้อมูลได้ ต้องรักษา temporal order เสมอ — Train ก่อน Test เสมอ 2 folds ทำให้ประเมินได้ว่าโมเดลทำงานได้ดีในช่วงเวลาต่างกัน: Fold1 test ปี 2019-2021, Fold2 test ปี 2022-2024 ช่วงวันที่เดียวกันทุกประการกับ LSTM ทำให้ผล metric เปรียบเทียบกันได้โดยตรง
-------------	---

ขั้นตอนที่ 15 Split + Feature Selection (= LSTM Cell 40+50)	
ทำอะไร	Cell 40: วน loop 132 สถานี slice DataFrame ตามช่วงวันที่ → ได้ Train/Val/Test list 6 ชุด จัดเก็บใน fold = [[Train1,Test1,Val1], [Train2,Test2,Val2]] Cell 50: apply select_feature_f() in-place ทั้ง 6 list — ตัดคอลัมน์ที่ไม่ใช้ออก เหลือ 9 คอลัมน์
ทำเพื่ออะไร	แยก Train/Val/Test ก่อน แล้วค่อยเลือก feature ที่หลัง ทำให้ทุก split มี feature เหมือนกัน ใช้ in-place modification เหมือน LSTM Cell 50 ทุกประการ — สำคัญสำหรับให้ index ตรงกัน โครงสร้าง fold[fold_index][split][station] ทำให้เข้าถึงข้อมูลแต่ละ fold/split/สถานีได้ง่าย

ขั้นตอนที่ 16 ระบุสถานีใน Cluster 6 (= LSTM Cell 53)	
ทำอะไร	วน loop ทุกสถานีใน Y_rainfall_normalized_df.columns เลือกเฉพาะที่ node['quality']==1 AND node['cluster']==6 ผล: cluster_station = [17, 30, 31, 34, 35, 36, 37] station_codes = ['351201','376201','376202','376401','378201','379201','379401']
ทำเพื่ออะไร	บางสถานีมี quality=0 หมายถึงข้อมูลไม่น่าเชื่อถือ (เช่น sensor เสีย, ข้อมูลหายบ่อย) ต้องคัดออก เราต้องการเฉพาะสถานีใน Cluster 6 ซึ่งตั้งอยู่ในภาคเหนือตอนล่าง-กลาง (ตาก, พิษณุโลก, เพชรบูรณ์)

ส่วนที่ 3 สร้าง Dataset สำหรับโมเดล

ขั้นตอนที่ 17 สร้าง Sliding Window Dataset (แทน LSTM Cell 66)	
ทำอะไร	ฟังก์ชัน <code>build_sliding_windows(fold_index, split_type)</code> ทำงานดังนี้: - นับ $n_steps = \text{len}(\text{data}[s]) - \text{window_size} - \text{horizon} + 1$ - วน loop j จาก 0 ถึง $n_steps-1$ (เลื่อนทีละ 1 เดือน): สำหรับแต่ละสถานีใน <code>cluster_station</code>: $x_window = \text{df.iloc}[j:j+24, :].\text{values.flatten}()$ → vector 216 มิติ $y_target = \text{df.iloc}[j+24:j+36, -1].\text{values}$ → rainfall 12 เดือน - รวมเป็น matrix X ($n_rows \times 216$) และ y ($n_rows \times 12$) Train Fold1: 421 snapshots $\times$ 7 สถานี = 2,947 rows Test Fold1: 1 snapshot $\times$ 7 สถานี = 7 rows (window=36 เดือนพอดี)
ทำเพื่ออะไร	LSTM รับ sequential data ทีละ time step ผ่าน HeteroData tensor — Decision Tree ต้องการ flat matrix การ flatten window ทำให้โมเดลเห็น pattern ของทุก feature ตลอด 24 เดือนย้อนหลังพร้อมกันใน 1 row Loop เหมือน LSTM Cell 66 ทุกประการ: $\text{range}(\text{len}(\text{fold}[fi][split][s]) - \text{window_size} - \text{horizon} + 1)$ Test มี 1 snapshot เพราะ window=36=24+12 พอดี — LSTM ก็มี 1 snapshot เหมือนกัน

ส่วนที่ 4 กำหนด Metrics

ขั้นตอนที่ 18 กำหนดฟังก์ชัน Metric (= LSTM Cell 68+72)	
ทำอะไร	<code>nse_score(obs, pred)</code>: Nash-Sutcliffe Efficiency สูตร: $\text{NSE} = 1 - \frac{\sum(\text{obs}-\text{pred})^2}{\sum(\text{obs}-\text{mean}(\text{obs}))^2}$ เหมือน LSTM Cell 72 ทุกบรรทัด <code>accuracy2D_classic_np(y_pred, y_true)</code>: สูตร: $1 - \min(1, \text{pred}-\text{true} / \text{true})$ สำหรับ $\text{true} \neq 0$ สำหรับ $\text{true} = 0$: $1 - \min(1, 2 \text{pred}-\text{true} / \text{pred}+\text{true})$ เหมือน LSTM Cell 68 ทุกบรรทัด

ทำเพื่ออะไร

NSE เป็น metric มาตรฐานใน hydrology: $NSE=1$ คือสมบูรณ์แบบ, $NSE=0$ คือเทียบเท่า mean, $NSE<0$ คือแย่กว่า mean

accuracy2D_classic วัดทั้งทิศทางและขนาด (ต่างจาก sign-based ที่วัดแค่ทิศ)

ใช้สูตรเดียวกับ LSTM ทำให้ตัวเลข accuracy เปรียบเทียบกันได้โดยตรง

ส่วนที่ 5 เทรน ประเมินผล และแสดงกราฟ

ขั้นตอนที่ 19 Train & Evaluate DecisionTreeRegressor

ทำอะไร	วน loop 2 folds: - สร้าง DecisionTreeRegressor(max_depth=None, random_state=0) - model.fit(X_train, y_train) — เรียนรู้ decision rules จาก training data - model.predict() บน train, val, test - คำนวณ metrics ครบทุกตัวสำหรับ 3 sets: Train: Acc, MSE, RMSE, R², NSE Val: Acc, MSE, RMSE, R², NSE + per-horizon acc Test: Acc, MSE, RMSE, R², NSE + per-horizon acc - บันทึกโมเดลและ predictions ลง datasets dict
ทำเพื่ออะไร	fit() ครั้งเดียวก็ได้โมเดลที่พร้อมใช้งานเลย ต่างจาก LSTM ที่ต้องวน loop 300 epochs multi-output regression: y มี 12 คอลัมน์ — DecisionTree รองรับโดยตรง ทำนายทุก horizon พร้อมกัน คำนวณ metrics ครบทั้ง 3 sets เพื่อตรวจ overfitting: ถ้า train ดีมาก แต่ val/test แย่ = overfit

ผลลัพธ์จาก Cell 19

Metric	DT Train	DT Val F1	DT Test F1	DT Val F2	DT Test F2
Acc (%)	100.00 ⚠	40.63	42.48	46.77	46.94
MSE	0.0000	0.6553	1.0036	0.7859	0.5203
RMSE	0.0000	0.8095	1.0018	0.8865	0.7213
R ²	1.0000	-6.529	-3.535	-9.619	-4.647
NSE	1.0000	0.1455	0.3353	0.3400	0.3803

⚠ Train Acc=100%, R²=1.0 คือสัญญาณ overfitting — โมเดล memorize training data ทั้งหมด
แก้ด้วยการตั้ง max_depth=5 ถึง 15

ขั้นตอนที่ 20 บันทึก Metrics ลง CSV

ทำอะไร	สร้างโฟลเดอร์ metric_results_DT/ ถ้ายังไม่มี บันทึก results_df ลงไฟล์ predict_DecisionTree_V1_cluster=6_numfeatures=8_window=24.csv
ทำเพื่ออะไร	บันทึก metrics ทุกตัวไว้ใช้เปรียบเทียบกับ LSTM ในภายหลัง ชื่อไฟล์ format เดียวกับ LSTM (predict_algo_cluster_numfeatures_window) ทำให้ทราบ context ทันทิโดยไม่ต้องเปิดไฟล์

ขั้นตอนที่ 21 กราฟ Forecast vs Actual — All 12 Horizons

ทำอะไร	วน loop 2 folds × 7 สถานี แสดง subplot ต่อสถานี แกน X = Month +1 ถึง +12 (12 horizon steps) เส้นน้ำเงิน (b-o) = ค่าจริง (Observed) เส้นแดง (r--s) = ค่าที่โมเดลทำนาย (Predicted) Title บอกว่าพยากรณ์จากเดือนใด (pred from YYYY-MM)
ทำเพื่ออะไร	เพราะ Test มีเพียง 1 snapshot กราฟจึงแสดง 12 จุด = ค่าพยากรณ์ Month +1 ถึง +12 จากจุดเริ่มต้นเดียว การ plot ทุก horizon ทำให้เห็นว่าโมเดลทำนายแม่นยำในเดือนไหน และเริ่มเพี้ยนเมื่อ lead time นานขึ้น การ plot แบบเดิม (loop ตาม snapshot) จะได้เพียง 1 จุดต่อกราฟ ทำให้แกน X เป็น float — แก้แล้วด้วยการ plot ตาม horizon

ขั้นตอนที่ 22 Feature Importance — Top 20

ทำอะไร	ดึงค่า model.feature_importances_ (vector ขนาด 216) สร้างชื่อ feature ในรูปแบบ ชื่อ_col_t-N เช่น 'ONI_lag1_t-1' คือ ONI_lag1 ณ เดือนล่าสุด เรียง importance จากมากไปน้อย แสดง bar chart Top 20
--------	--

ทำเพื่ออะไร	Feature importance ของ Decision Tree บอกว่า feature ใดถูกเลือกไปใช้แบ่ง node มากที่สุด ซึ่งสะท้อนว่า ดัชนีภูมิอากาศตัวใดและช่วงเวลาใดในหน้าต่าง 24 เดือน มีผลต่อการทำนายฝนมากที่สุด เป็นข้อได้เปรียบสำคัญของ Decision Tree เหนือ LSTM ซึ่งเป็น black box ที่ความยาก
-------------	--

ขั้นตอนที่ 23 Per-Horizon Accuracy (Month +1 → +12)	
ทำอะไร	ดึง acc_month1 ถึง acc_month12 จาก results_df สำหรับ 2 folds แสดง bar chart เคียงกัน 2 subplots (Fold 1 และ Fold 2) เส้นประสีเหลืองแสดง 50% baseline
ทำเพื่ออะไร	กราฟนี้แสดงให้เห็นว่า accuracy ลดลงตามระยะเวลาพยากรณ์อย่างไร ช่วง Month +4 ถึง +6 accuracy ต่ำทั้ง LSTM และ DT เพราะเป็นช่วงเปลี่ยนฤดูกาลที่ฝนแปรปรวนมาก เปรียบเทียบกับกราฟเดียวกันของ LSTM ทำให้เห็นว่าโมเดลใดรักษาความแม่นยำได้นานกว่าเมื่อ lead time เพิ่มขึ้น

สรุปภาพรวม Pipeline ทั้งหมด 23 Cell

Cell	หัวข้อ	ทำอะไร	ทำเพื่ออะไร (สั้น)
1	Import Libraries	นำเข้า library ทั้งหมด	ประกาศ dependency ก่อนใช้งาน
2	Hyperparameters	ตั้งค่า window=24, horizon=12, cluster=6 ฯลฯ	รวมค่าคงที่ไว้จุดเดียว แก้ไขง่าย
3	Path ไฟล์	กำหนด path ของ CSV ทั้ง 4	รองรับหลาย environment (Local/Colab)
4	โหลดข้อมูลดิบ	อ่าน CSV 4 ไฟล์	เตรียมข้อมูลดิบก่อนทุกขั้น
5	Lag Features	สร้าง lag 14 คอลัมน์ ก่อน normalize	จับ delayed effect ของดัชนีภูมิอากาศ
6	Cleanse Y	เติม NaN ด้วย monthly median	ML ไม่รับ NaN — ใช้ median รายเดือนเหมาะกับ seasonality
7	Filter Date #1	ตัดข้อมูลก่อน 1982-01-01	ข้อมูลก่อน 1982 ไม่สมบูรณ์
8	Filter Date #2	reset start = Y.min() แล้ว filter ซ้ำ	เหมือน LSTM Cell 14 ทุกประการ
9	Set DATE Index	ตั้ง DATE เป็น index	Slice ข้อมูลตามวันที่ได้ง่าย
10	Normalize	StandardScaler fit_transform()	ทุก feature มี scale เดียวกัน
11	Month sin/cos	เพิ่ม circular encoding ของเดือน	แก้ปัญหา discontinuity เดือน 12→1
12	Per-Station Table	สร้าง DataFrame แยกต่อสถานี 132 ตัว	รองรับ loop ต่อสถานีในขั้นถัดไป
13	Feature Selection	เลือก 8 features จาก lookup_table	ตัด noise เหลือเฉพาะ signal จริง
14	กำหนดช่วงวันที่	คำนวณ start/end date ทั้ง 6 ช่วง	Time-series CV ไม่ shuffle
15	Split + Feat. Sel.	slice DataFrame + ตัดคอลัมน์	สร้าง train/val/test แยก fold
16	Cluster Stations	ระบุ index ของ 7 สถานี quality=1	คัดสถานีที่ข้อมูลน่าเชื่อถือ
17	Sliding Window	flatten 24×9=216 มิติ per row	แปลง time-series เป็น tabular สำหรับ DT
18	Metric Functions	นิยาม NSE และ accuracy2D_classic	ใช้สูตรเดียวกับ LSTM เปรียบเทียบได้

19	Train & Evaluate	fit() + metrics ครบ 3 sets	เทรนโมเดลและวัดผล
20	บันทึก CSV	save results_df	เก็บผลเปรียบเทียบกับ LSTM
21	กราฟ Forecast	plot 12 horizons ต่อสถานี	เห็นภาพ pred vs actual
22	Feature Importance	bar chart Top 20 feature	ตีความว่า feature ไດสำคัญ
23	Per-Horizon Acc	bar chart acc ต่อ lead time	เห็น decay ของความแม่นยำ